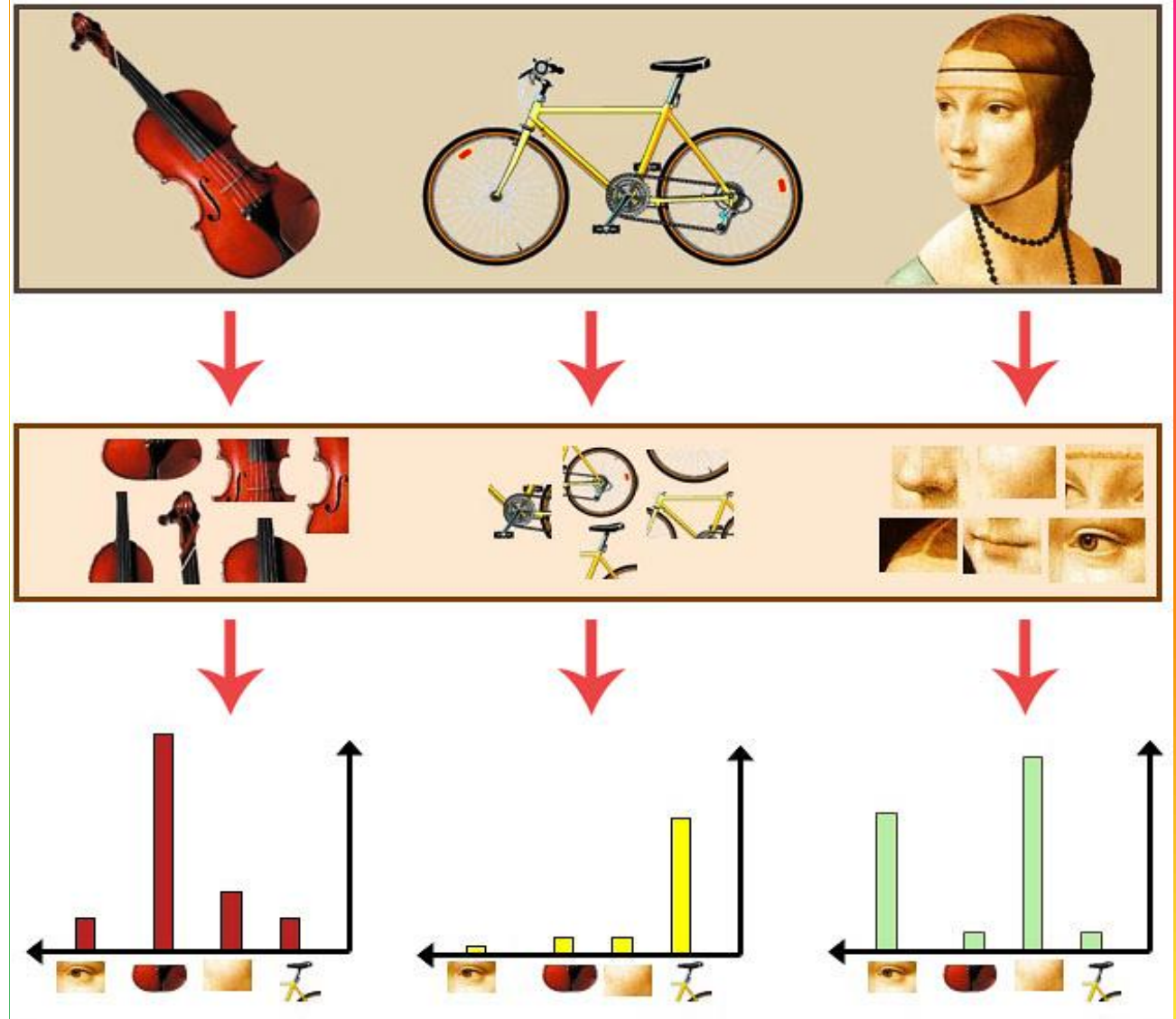


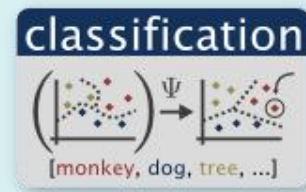
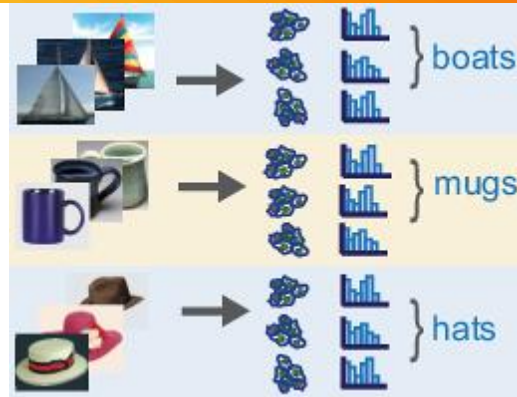


Università
degli Studi
di Palermo

BAG OF VISUAL WORDS

Pietro Sciacca
Computer Vision
A.Y. 2025-2026





Airplanes	         
Motorbikes	         
Faces	         
Wild Cats	         
Leaves	         
People	         
Bikes	         

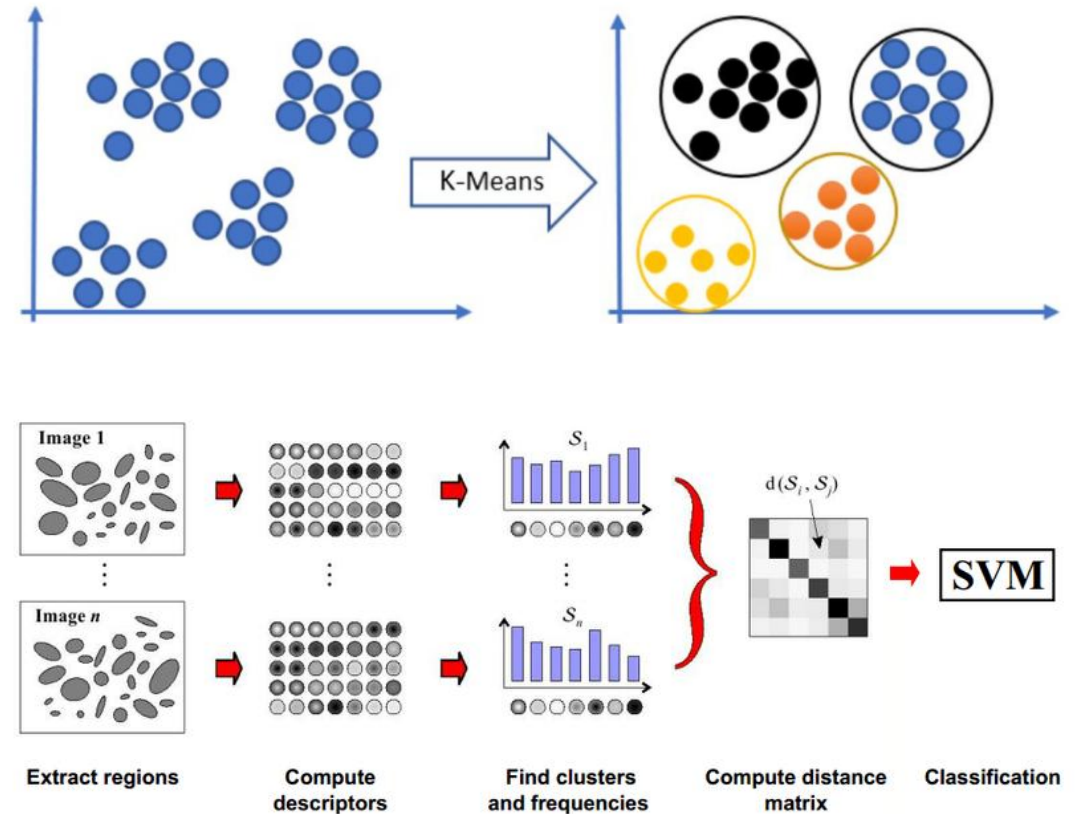
In the Bag of Words model, a document is represented by the frequency of its words, which is useful for tasks like text retrieval or classification.

Similarly, an image can be described by the **frequency of local visual patterns**: it ignores where features are located in the image and focuses entirely on what visual elements are present.

Dictionary

Images contain a variable number of descriptors, for content comparison we need a visual dictionary that quantizes these vectors into a **fixed vocabulary of descriptors**.

Millions of SIFT descriptors are extracted from the training set and **partitioned into distinct groups** using K-Means clustering; the centroids of these groups define the visual words that compose the final dictionary, providing a baseline to image comparison.



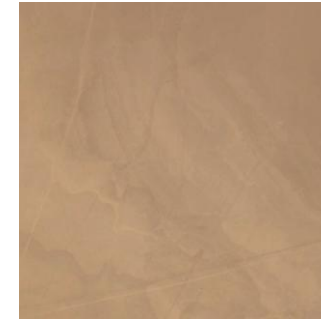
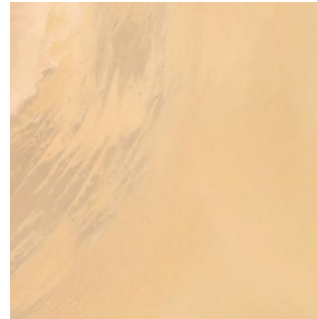


```
Processing folder: Airport
Processing folder: BareLand
Error no descriptor: ../AID/BareLand/bareland_207.jpg
Error no descriptor: ../AID/BareLand/bareland_43.jpg
Error no descriptor: ../AID/BareLand/bareland_62.jpg
Processing folder: BaseballField
Processing folder: Beach
Processing folder: Bridge
Processing folder: Center
Processing folder: Church
Processing folder: Commercial
Processing folder: DenseResidential
Processing folder: Desert
Error no descriptor: ../AID/Desert/desert_106.jpg
Error no descriptor: ../AID/Desert/desert_11.jpg
Error no descriptor: ../AID/Desert/desert_155.jpg
Error no descriptor: ../AID/Desert/desert_126.jpg
Error no descriptor: ../AID/Desert/desert_207.jpg
Error no descriptor: ../AID/Desert/desert_218.jpg
Error no descriptor: ../AID/Desert/desert_210.jpg
Error no descriptor: ../AID/Desert/desert_208.jpg
Error no descriptor: ../AID/Desert/desert_223.jpg
Error no descriptor: ../AID/Desert/desert_2.jpg
Error no descriptor: ../AID/Desert/desert_290.jpg
Error no descriptor: ../AID/Desert/desert_300.jpg
Error no descriptor: ../AID/Desert/desert_295.jpg
Error no descriptor: ../AID/Desert/desert_285.jpg
Error no descriptor: ../AID/Desert/desert_36.jpg
Error no descriptor: ../AID/Desert/desert_29.jpg
Error no descriptor: ../AID/Desert/desert_6.jpg
Error no descriptor: ../AID/Desert/desert_46.jpg
Error no descriptor: ../AID/Desert/desert_89.jpg
Error no descriptor: ../AID/Desert/desert_60.jpg
Error no descriptor: ../AID/Desert/desert_92.jpg
Error no descriptor: ../AID/Desert/desert_47.jpg
Error no descriptor: ../AID/Desert/desert_94.jpg
Error no descriptor: ../AID/Desert/desert_87.jpg
Error no descriptor: ../AID/Desert/desert_88.jpg
Processing folder: Farmland
Processing folder: Forest
Processing folder: Industrial
Processing folder: Meadow
Processing folder: MediumResidential
Processing folder: Farmland
Processing folder: Forest
Processing folder: Industrial
Processing folder: Meadow
Processing folder: MediumResidential
Processing folder: Mountain
Processing folder: Park
Processing folder: Parking
Processing folder: Playground
Processing folder: Pond
Processing folder: Port
Processing folder: RailwayStation
Corrupt JPEG data: premature end of data segment
Processing folder: Resort
Processing folder: River
Processing folder: School
Processing folder: SparseResidential
Processing folder: Square
Processing folder: Stadium
Processing folder: StorageTanks
Processing folder: Viaduct
```

For this purpose, I've extracted nearly **2 million SIFT descriptors**, 200 descriptors for image, from the dataset AID (<https://www.kaggle.com/datasets/jiayuanchengala/aid-scene-classification-datasets>).

Descriptor Dataset: (1951578, 128)

Here some images with no descriptors:



```
Start K-means Clustering, K=50
Clustering 1951578 points in 128D to 50 clusters, redo 8 times, 100 iterations
Preprocessing in 0.23 s
Outer iteration 0 / 8
  Iteration 99 (34.85 s, search 27.56 s): objective=587874 imbalance=1.049 nsplit=0
Objective improved: keep new clusters
Outer iteration 1 / 8
  Iteration 99 (69.83 s, search 55.38 s): objective=586597 imbalance=1.043 nsplit=0
Objective improved: keep new clusters
Outer iteration 2 / 8
  Iteration 99 (103.99 s, search 82.83 s): objective=586562 imbalance=1.061 nsplit=0
Objective improved: keep new clusters
Outer iteration 3 / 8
  Iteration 99 (136.10 s, search 108.65 s): objective=588388 imbalance=1.049 nsplit=0
Outer iteration 4 / 8
  Iteration 99 (168.77 s, search 134.91 s): objective=587345 imbalance=1.041 nsplit=0
Outer iteration 5 / 8
  Iteration 99 (201.76 s, search 161.21 s): objective=585765 imbalance=1.058 nsplit=0
Objective improved: keep new clusters
Outer iteration 6 / 8
  Iteration 99 (235.79 s, search 188.33 s): objective=588149 imbalance=1.064 nsplit=0
Outer iteration 7 / 8
  Iteration 99 (270.48 s, search 216.06 s): objective=586344 imbalance=1.059 nsplit=0
Clustering completed: 270.72 s
```

```
Start K-means Clustering, K=1000
Clustering 1951578 points in 128D to 1000 clusters, redo 8 times, 100 iterations
Preprocessing in 0.23 s
Outer iteration 0 / 8
  Iteration 99 (161.00 s, search 155.26 s): objective=430977 imbalance=1.077 nsplit=0
Objective improved: keep new clusters
Outer iteration 1 / 8
  Iteration 99 (320.25 s, search 308.87 s): objective=430564 imbalance=1.072 nsplit=0
Objective improved: keep new clusters
Outer iteration 2 / 8
  Iteration 99 (473.96 s, search 457.02 s): objective=430724 imbalance=1.072 nsplit=0
Outer iteration 3 / 8
  Iteration 99 (625.88 s, search 603.40 s): objective=430661 imbalance=1.065 nsplit=0
Outer iteration 4 / 8
  Iteration 99 (777.44 s, search 749.47 s): objective=430815 imbalance=1.072 nsplit=0
Outer iteration 5 / 8
  Iteration 99 (951.49 s, search 918.07 s): objective=430656 imbalance=1.067 nsplit=0
Outer iteration 6 / 8
  Iteration 99 (1172.75 s, search 1133.95 s): objective=430678 imbalance=1.067 nsplit=0
Outer iteration 7 / 8
  Iteration 99 (1396.40 s, search 1352.22 s): objective=430719 imbalance=1.066 nsplit=0
Clustering completed: 1396.64 s
```

I normalized the descriptors using the **L₂ norm** to counteract OpenCV's internal integer scaling (x512), ensuring all visual features have equal scale and length 1.

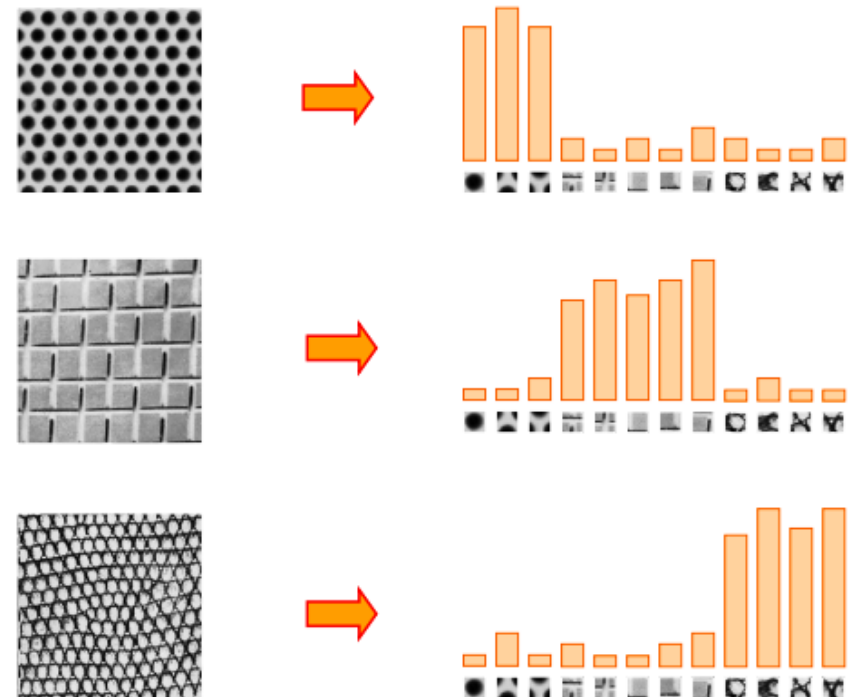
Next I ran K-Means clustering with **dictionary sizes of 50, 100, 500, and 1000 visual words**. Testing four vocabularies of different sizes allow to evaluate how prediction accuracy scales with vocabulary size.

To handle the computational load of clustering, I leveraged **FAISS** (Facebook AI Similarity Search), a highly optimized library designed for high-performance clustering.

Representation of images

For each image, its extracted SIFT descriptors are compared against the words of the visual dictionary, and using a Nearest Neighbor search like Euclidean distance, **every 128D descriptor is assigned to its closest visual word.**

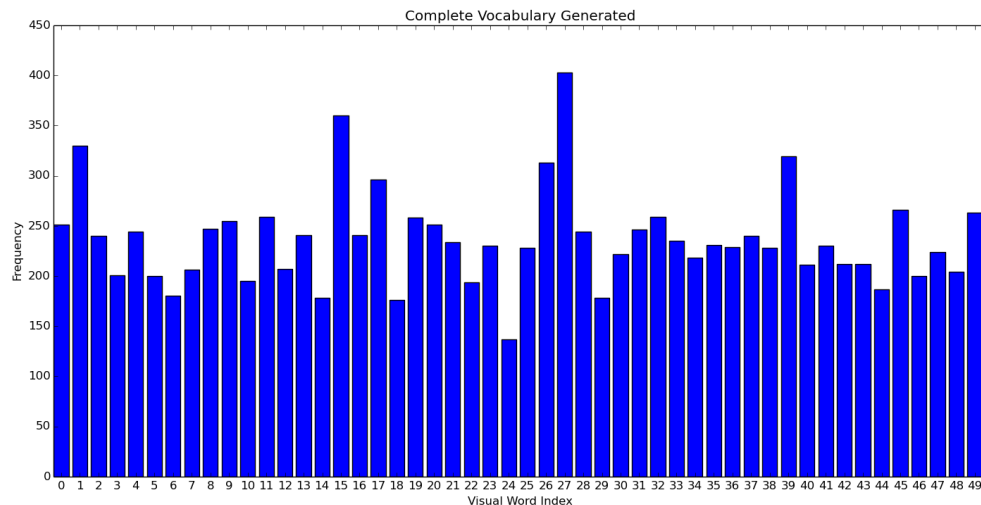
The raw histogram is **normalized with L_2 norm to ensure scale invariance**, avoiding that two image of the same scene result different because of the resolution.





Features were extracted from the training dataset composed of 2100 images (<https://www.kaggle.com/datasets/abdulhasibuddin/uc-merced-land-use-dataset>) to construct the **final histograms for each of the four dictionary scales** ($K=50, 100, 500, 1000$).

This step transformed the raw image into a fixed-length frequency vector, which **count the occurrence of every visual words** and provide a standardized numerical input for classification models.

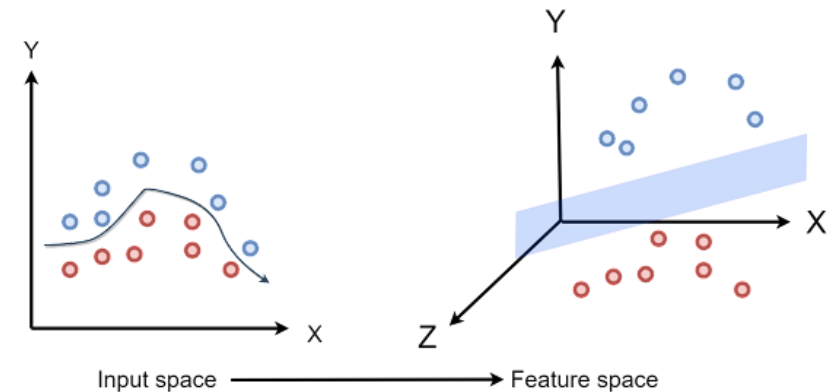
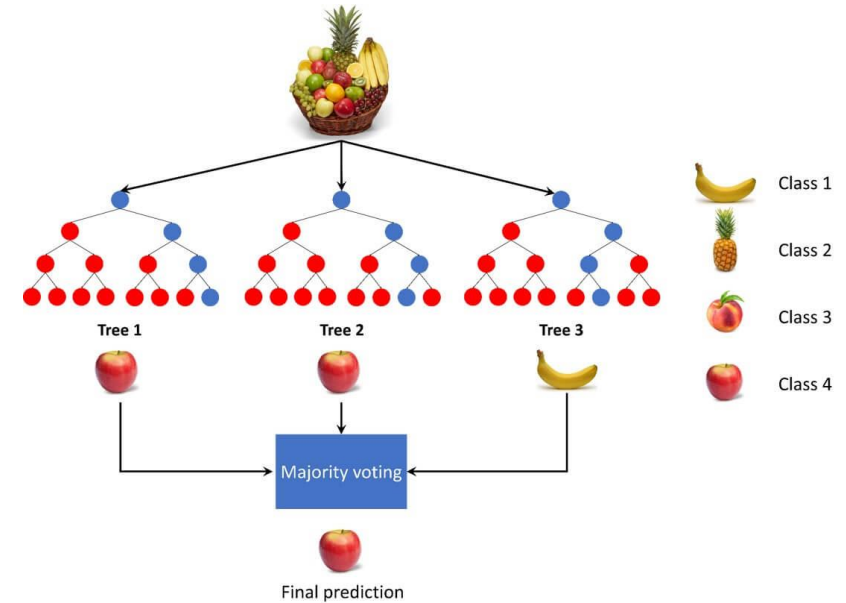


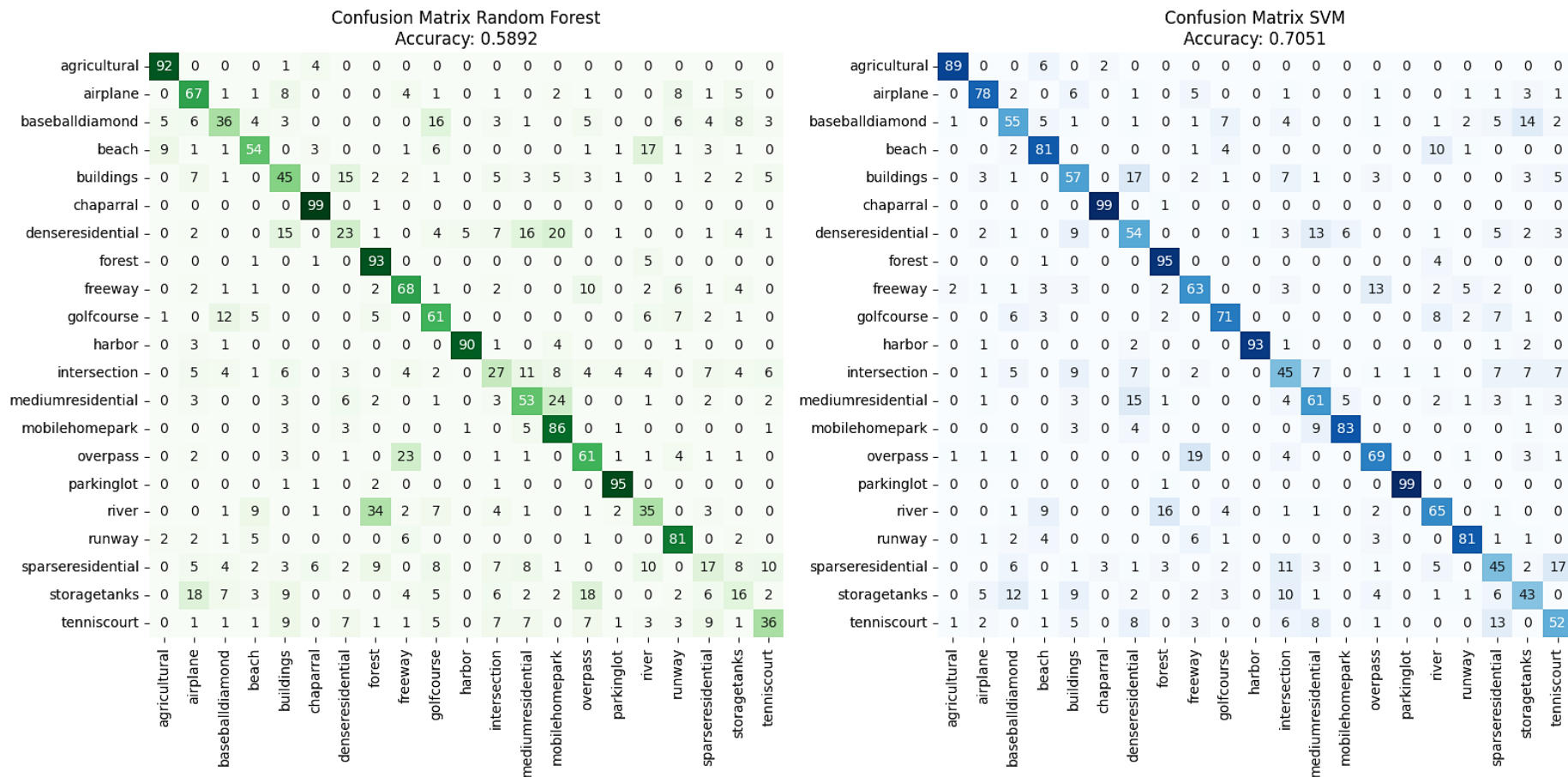
Classification

The dataset used for training is UC Merced Land Use Dataset, composed of **21 distinct classes of 100 images each**.

The histogram extracted before are used as description of the image, while the labels represent the different objects protraied in the scene.

The classification models evaluated are **Random Forest Classifier** and **Support Vector Machine (SVM)** with RBF kernel.

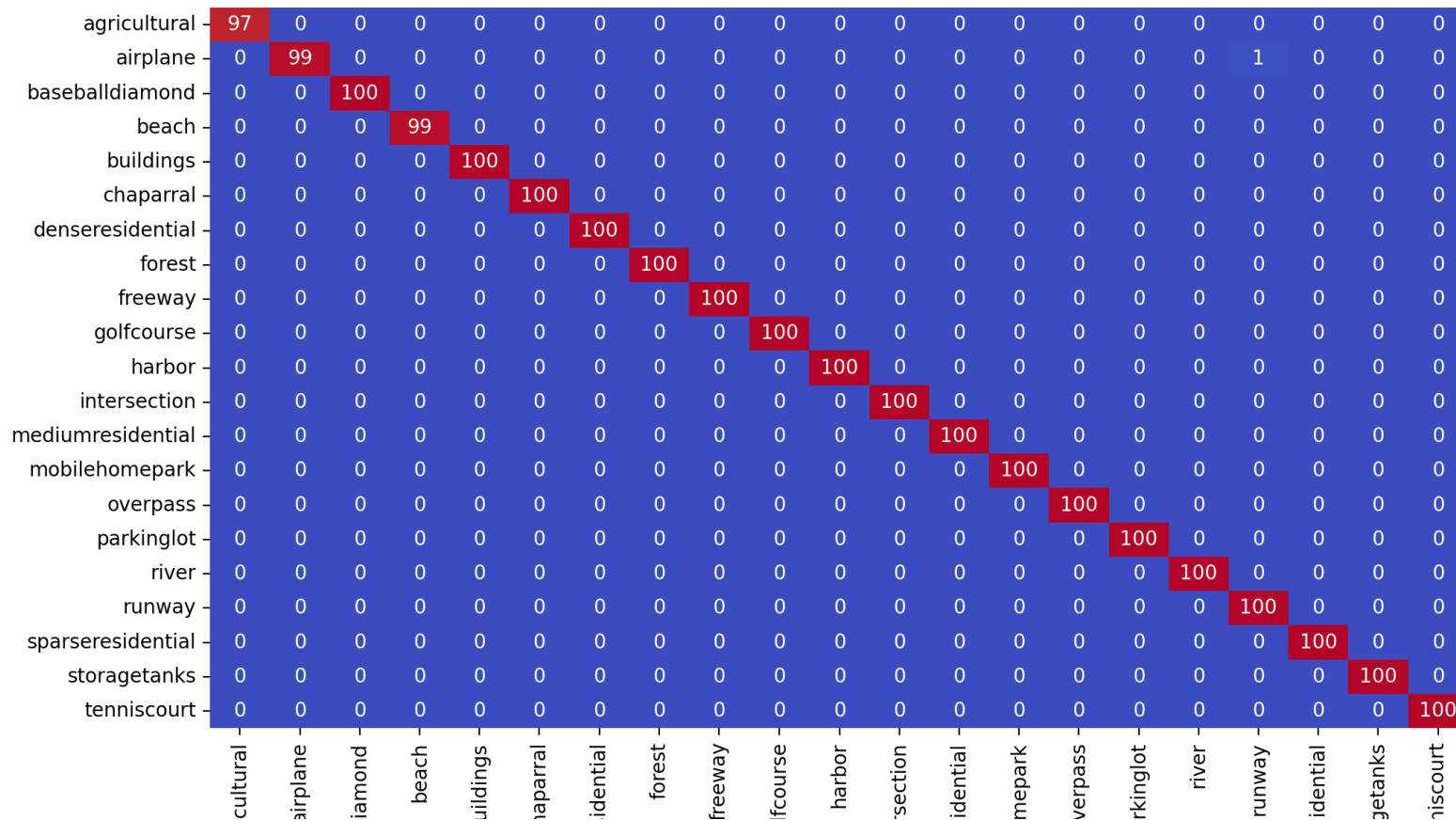




The image show results on 3-fold validation for dictionary dimension K=500:

- SVM outperforms Random Forest across all dictionary sizes.
- Accuracy increases by an average of 5% when moving from K=50 to 100, and from K=100 to 500, the performance difference between K=500 and K=1000 is infinitesimal.

SVM with K=500 is selected as the optimal model, providing balance between accuracy and computational efficiency.



The confusion matrix results correspond to the **final SVM model**, after training on all the examples of the dataset.

Despite the near-perfect accuracy, the model suffers from **overfitting** and struggles to generalize unseen images.

This is primarily due to the small size of training dataset and the handcrafted descriptors from every image.

To overcome these limitations and improve generalization despite the small training set, handcrafted SIFT features can be replaced with deep features extracted from a **pre-trained Convolutional Neural Network**, like VGG or ResNet.

Here we can see some examples of classification errors on unseen images, on a test set built from web's images, available in the folder "images".



Label: agricultural

Classification: runway



Label: harbor

Classification: mobilehomepark



Possible labels: airplane, forest

Classification: golfcourse



Label: baseballdiamond

Classification: runway



Label: beach

Classification: chaparral



Label: runway

Classification: runway